eOffice-Style File & Workflow Management System

A Technical Blueprint — Architecture, Build Plan, Technology Stack & Scaling Strategy

Reference system studied: services.eoffice.gov.in (NIC eOffice, Government of India)

Prepared By: Shahnawaz Hussain

1. What eOffice Actually Is

eOffice is a digital workplace suite built by India's National Informatics Centre (NIC) that replaces the paper file, the physical folder that moves from desk to desk for noting, approval, and record-keeping — with an electronic equivalent. It is not a document editor and not a citizen-facing portal; it automates the internal machinery of an organisation: how a request enters, how it is discussed and annotated by a chain of people, how a decision is made, and how the record is filed away for audit and retrieval.

The public downloads page you referenced distributes the client-facing pieces of this suite: the eFile desktop/web module, mobile apps, digital signature (DSC) utilities, and browser plugins needed to sign and view files. The real product, though, is the workflow engine and repository sitting behind those clients.

1.1 Core Modules

Module	Purpose

eFile	The heart of the system. Handles Receipts (incoming letters/requests), File creation, Noting (internal comments/decisions attached to a file), Movement (routing a file up/down a hierarchy), Dispatch (outgoing correspondence), and DSC-based digital signing.
KMS (Knowledge Management System)	Central document repository with version history, OCR-based full-text search (including text inside scanned images), and role-based access.
CAMS	Internal collaboration and messaging tied to files — comments, mentions, notifications.
SPARROW	Personnel records (HR-style module) — used for leave, tour, and service-book style data.

1.2 Why It Matters

Strip away the government-specific vocabulary (Tappal, DAK, CSMOP) and what remains is a generic, extremely common enterprise pattern:

- A structured intake mechanism for requests/documents (Receipts)
- A stateful "case" or "file" object that things get attached to
- A configurable approval chain / routing engine (who sees it next)
- An immutable audit trail of every action, comment, and movement
- A searchable document repository with versioning
- Dashboards for pendency

This is the same shape as help-desk, loan-origination workflows, HR approval chains, legal contract-review pipelines, and procurement approval systems. Building this for the private sector means building a generic workflow + document-management engine and then configuring it per use case — which is a much bigger and more reusable win than copying eOffice module-for-module.

2. How the System Works — The Lifecycle of a File

Everything in the eOffice revolves around one lifecycle. Understanding it precisely is the key to designing the data model correctly.

2.1 The Six Stages

1. **Receipt (Intake):** A document/request enters the system — uploaded, emailed in, or scanned. It is registered and stamped with a unique tracking number (diary number).
2. **File Creation / Attachment:** The receipt is attached to a new or existing "file" — a persistent case folder that accumulates every related document over its lifetime.
3. **Noting:** Each person in the chain adds a timestamped, immutable note (their opinion/decision) directly on the file — never edited afterward, only appended to.

4. **Movement:** The file is routed to the next person, either along a pre-defined hierarchy or an ad-hoc reviewer chosen by the sender. Every hop is logged with a timestamp and actor.
5. **Approval / Disposal:** A person with sufficient authority approves, rejects, or sends the file back down the chain. Digital signatures (DSC/e-sign) can be attached at this point for legal validity.
6. **Dispatch / Archival:** Once resolved, an outward communication may be generated and sent, and the file is archived — but never deleted, only closed. It remains searchable forever.

> **The one non-negotiable rule**
> A file's history is append-only. Nothing is ever overwritten or deleted — corrections happen by adding a new note, never by editing an old one. This is what makes the audit trail trustworthy, and it should be a hard architectural constraint, not just a UI convention.

2.2 Roles and Permissions

Movement only works because every user has a place in a hierarchy or a role: Dealing Assistant → Section Officer → Under Secretary → Director → Secretary (in government terms). In a private-sector clone this becomes configurable org roles: Requester → Reviewer → Approver → Admin, with the routing chain defined per workflow/department rather than hard-coded.

3. How To Build It — System Architecture

Below is a pragmatic architecture for a private-sector version, sized so a small team (or a solo backend engineer) can build a working MVP in 8–12 weeks and grow it into a multi-person SaaS product afterward.

3.1 High-Level Component View

- **Client apps:** Web app (primary), optional mobile app for approvals-on-the-go.
- **API layer:** REST — authentication, request validation, rate limiting.
- **Workflow engine:** A dedicated service that owns "file state", routing rules, and transitions. This is the module worth investing the most design time in.
- **Document store:** Object storage (S3-compatible) for the actual files + a metadata index in Postgres.
- **Search index:** Full-text + OCR search, separate from the primary DB.
- **Notification service:** Email/push/in-app alerts on movement, SLA breach, mentions.
- **Audit log store:** Append-only event log, ideally separate from the mutable operational DB.
- **Digital signature integration:** Third-party e-sign provider or a PKI-based DSC integration for legal validity.

3.2 Core Data Model

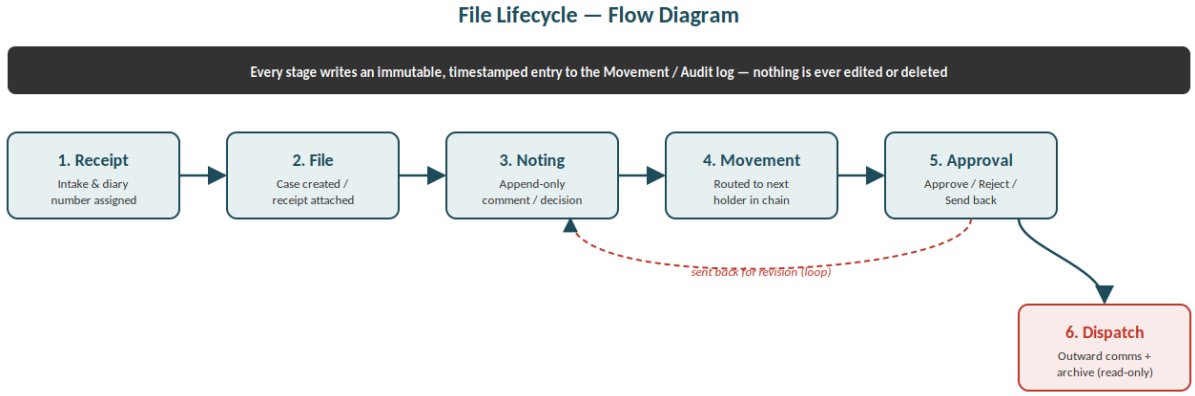
These are the entities that matter; everything else is supporting detail.

Entity	Key fields	Notes

Organization / person	id, name, plan, settings	Root of multi-tenancy — every other table scopes to this.
User	id, org_id, role, hierarchy_position	Hierarchy or role drives who a file can move to.
Receipt	id, org_id, source, received_at, diary_no	Point of intake; immutable once created.
File (Case)	id, org_id, status, current_holder_id, sla_due_at	The central stateful object. status is an enum driven by the workflow engine.
Note	id, file_id, author_id, body, created_at	Append-only. Never updated or deleted.
Movement	id, file_id, from_user, to_user, action, created_at	One row per hop. This IS the audit trail for routing.
Document	id, file_id, storage_key, version, checksum	Metadata pointer to object storage; versions are new rows, not overwrites.
WorkflowDefinition	id, org_id, steps_json	Configurable per department/use-case — this is what makes it reusable beyond one org.

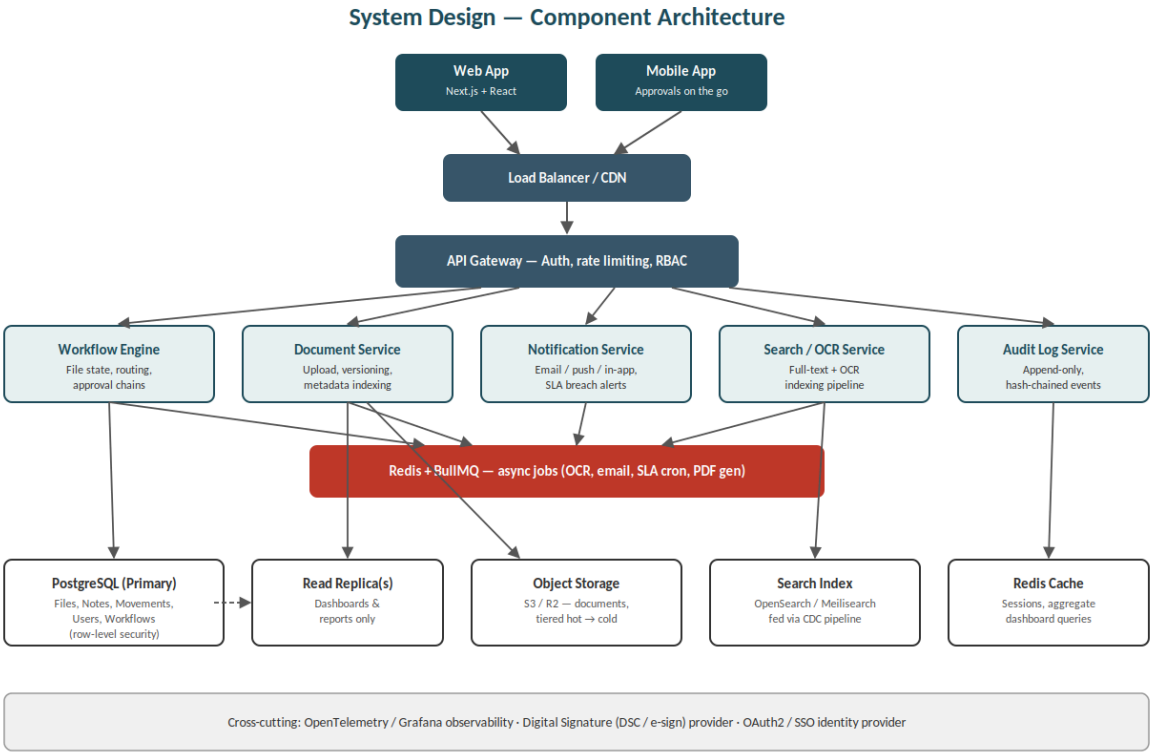
Diagrams — Flow & System Design

A. File Lifecycle — Flow Diagram



The straight path moves a file forward one stage at a time; the dashed red loop is a rejection/send-back, which re-enters at Noting rather than restarting the whole lifecycle. Everything above is mirrored by a permanent row in the audit log, which is what makes disposal rates and SLA breaches auditable after the fact.

B. System Design — Component Architecture



Clients talk to a single API gateway that owns auth, RBAC, and rate limiting. Behind it, four independently-scalable services (Workflow, Document, Notification, Search/OCR) share a Redis/BullMQ queue for anything that doesn't need to block the request. Writes land on the primary Postgres instance; dashboards and reports read from a replica so they never contend with file-movement traffic. Documents live in object storage, never in the relational DB, and the Audit Log service is deliberately drawn as its own box because it has a different durability requirement — append-only, tamper-evident — from the mutable operational data around it.

4. Security, Compliance & Audit Considerations

A workflow/records system's entire value proposition is trust in the record. Security is not a bolt-on here — it's the product.

- **Append-only audit log:** store Notes and Movements in a way that makes tampering detectable — e.g. hash-chaining rows (each row's hash includes the previous row's hash) or writing the audit stream to a write-once store.

- **Row-level access control:** Postgres Row-Level Security (RLS) enforced at the DB layer, not just in application code, so a bug in one endpoint can't leak cross-person data.
- **Encryption:** TLS in transit; at-rest encryption for the object store and DB
- **Digital signatures / non-repudiation:** for approvals that need legal weight, integrate real e-sign/PKI rather than a plain "Approved by X" text field.
- **Data retention:** define retention/archival policy per document class up front — "never delete" is the safe default but gets expensive; tiered storage (hot → cold/glacier) handles this economically.
- **Compliance mapping:** depending on target customers, map to relevant standards — SOC 2 for enterprise SaaS buyers, India's DPDP Act 2023 for personal data, ISO 27001 if selling to government-adjacent or BFSI clients.

5. Scaling Strategy — The Detailed Plan

5.1 Stage 1 — Single Org, Low Volume (MVP → first customers)

At this stage a single Postgres instance, a single app server, and synchronous request handling are enough. The only architectural decision that matters now is making tenancy explicit from day one (an org_id column on every table) — retrofitting multi-tenancy later is far more expensive than building it in from the start.

- Vertical scaling of a single Postgres + app server pair is sufficient.
- Use connection pooling even at this stage — file-movement workloads are bursty (everyone acts in the morning), and pooling avoids connection exhaustion cheaply.
- Move slow work (OCR, email, PDF generation) into BullMQ jobs immediately, even if volume doesn't require it yet — it keeps request latency flat as volume grows, and you already have the muscle memory for this from RailBook.

5.2 Stage 2 — Multiple Orgs, Growing Volume

This is where most of the real engineering work happens. The bottlenecks tend to appear in this order: read-heavy dashboard queries, document storage growth, and search latency.

- **Read replicas:** route dashboard/report queries (pendency counts, SLA reports) to a Postgres read replica so they never contend with the write path that moves files between people.
- **Caching:** cache expensive aggregate queries (org-level pendency counts, dashboard summaries) in Redis with short TTLs — these are read far more often than the underlying data changes.
- **Partition the audit/movement log:** Movement and Note tables grow forever and are append-only — partition by org_id or by month so old partitions can be moved to cheaper storage without touching hot data.
- **Externalize search:** once full-text queries start competing with transactional load, move search into OpenSearch/Meilisearch fed by a change-data-capture pipeline (Postgres logical replication → indexer) rather than querying Postgres directly.

- **Object storage lifecycle rules:** auto-tier documents older than N months to cold storage classes; this is the single biggest cost lever in a document-heavy system.

5.3 Stage 3 — Many persons, High Concurrency (SaaS scale)

At this point the system is serving many organizations concurrently, and the concerns shift from "is the DB fast enough" to "can one noisy person degrade everyone else's experience."

- **person isolation strategy:** decide explicitly between shared schema with org_id (cheapest, needs strict RLS), schema-per-person (moderate isolation, harder migrations), or database-per-large-person for your biggest enterprise customers who need it contractually.
- **Horizontal API scaling:** stateless app servers behind a load balancer, auto-scaled on CPU/queue depth; this only works cleanly if session state lives in Redis, not in-process.
- **Workflow engine as its own service:** once file-state transitions are a hot path, split the workflow engine out from the general CRUD API so it can be scaled and tuned independently (it has different load characteristics — bursty, transaction-heavy, latency-sensitive).
- **Queue-based backpressure:** under load spikes (e.g. month-end approvals across every department at once), let BullMQ queues absorb the burst rather than letting API request latency spike — the user sees "submitted, processing" instead of a timeout.
- **Database sharding (only if truly necessary):** shard by org_id once a single Postgres primary can't hold the write throughput of your largest persons combined — this is usually the last resort, not the first move, and most systems never actually need it if partitioning and read replicas are used well first.

5.4 Stage 4 — Multi-Region / Enterprise Compliance

Relevant once customers require data residency (e.g. an EU customer's documents must stay in the EU) or you need to survive a full regional outage.

- Region-pinned persons: each org's data lives in one designated region; a thin global routing layer directs requests to the correct region rather than replicating everything everywhere.
- Async cross-region replication for disaster recovery only — avoid synchronous cross-region writes, which will make every file movement slow for the sake of a rare failover scenario.
- This directly echoes your own CyPSi Lab research interest — correctness-preserving degradation in multi-region backends is exactly the right lens here: define what the system is allowed to sacrifice (e.g. slightly stale dashboard counts) during a partial regional outage so that the core file-movement path keeps working correctly rather than failing outright.

> **The one scaling principle that matters most**
>
> Scale the read path and the write path separately, and scale the audit/document store separately from both. A file-movement (write) needs strict consistency and low latency for one row; a pendency dashboard (read) can tolerate a few seconds of staleness; a document blob doesn't need to be near either of them. Treating all three as one "database problem" is the most common reason these systems degrade under load.

6. Where a our Version Can Improve on eOffice

eOffice was built in 2009 against government procedural requirements (CSMOP) and a hierarchical, single-track approval culture. A private-sector clone is free of those constraints and can fix several known friction points:

eOffice limitation	Our improvement
Rigid, hierarchy-bound routing (file must climb a fixed ladder)	Configurable workflow definitions per department — parallel approvals, conditional branches, auto-escalation on SLA breach
Heavy reliance on LDAP/government email for auth	Modern OAuth2/SSO, support for external partners/vendors as guest reviewers
Desktop-first UX carried over from 2009-era design	Mobile-first approvals, real-time updates via WebSockets instead of manual refresh
Search limited largely to metadata + basic OCR	Semantic/AI-assisted search, auto-summarization of long files, smart suggested-next-approver
One-size-fits-all module set	Multi-person SaaS: each customer configures only the modules/workflows relevant to them
On-prem/state-data-centre hosting model	Cloud-native, horizontally scalable, with a managed-hosting or self-host option for compliance-sensitive customers
Reporting is largely descriptive (pendency counts)	Predictive analytics — flag files likely to breach SLA before they do, based on historical movement patterns

7. Suggested MVP Scope (What To Actually Build First)

Given everything above, a realistic first build for validating the idea with 1–2 org:

- Single-person (multi-person-ready schema, but no billing/isolation complexity yet)
- Receipt intake → File → Note → Movement → Approve/Reject, with a fixed 3-role chain (Requester → Reviewer → Approver)
- Document upload to S3-compatible storage with basic versioning
- Email notifications on movement (BullMQ job)
- A single pendency dashboard: open files, average time-in-stage, overdue count